

Integrating GitLab with Moodle

Bachelor Thesis

Department : TIC

Program : Computer Science and Communications Systems

Specialization : Software Engineering

Jouve Léonard

21 July 2026

Proposed and supervised by:

Delgado Pamela

HEIG-VD

Preamble

This Bachelor's thesis (hereinafter referred to as BT) is carried out at the end of the study program, with the aim of obtaining the Bachelor of Science HES-SO degree in Engineering.

As an academic work, its content, without prejudice to its value, does not engage the responsibility of either the author, the Bachelor's thesis jury, or the School.

Any use, even partial, of this BT must comply with copyright law.

HEIG-VD
The Head of the Department

Yverdon-les-bains, 21 July 2026

Authentication

I, the undersigned, Jouve Léonard, hereby declare that I have completed this work independently and have not used any sources other than those expressly cited.

Jouve Léonard

Yverdon-les-bains, 21 July 2026

Abstract

Learning Management Systems (LMS) such as Moodle play a central role in organizing higher education courses. At the same time, in software development, Version Control Systems (VCS) such as GitLab have become essential tools for source code management, project coordination, and collaboration. In computer science education, these systems are also widely used for distributing assignments and evaluating practical work. However, despite their complementary roles, LMS and VCS platforms typically operate independently, resulting in fragmented workflows.

This project addresses the lack of integration between Learning Management Systems and version control systems in computer science education by introducing **Repository Assignment Workflow (RAW)**. **RAW** is an open-source, extensible Moodle plugin that combines LMS-based assignment management with Git-based development workflows. The system automates repetitive administrative tasks, including repository provisioning, group management, synchronization, solution publication, and grading processes. By reducing manual overhead, **RAW** enables instructors to focus more on teaching while providing students with a centralized environment for accessing assignments, source code, instructions, and feedback.

RAW is built around three key principles: LMS-native integration to reduce platform fragmentation, flexible automation instead of rigid workflows, and a modular open-source architecture that supports extensibility. The evaluated existing solutions do not provide an equivalent combination of these characteristics, as they often lack native LMS integration, rely on proprietary ecosystems, or require significant manual intervention.

The current dependence on proprietary platforms raises long-term sustainability concerns, as changes in availability, licensing models, or strategic direction may disrupt established educational workflows. The announced discontinuation of GitHub Classroom on 28 August 2026 further illustrates these risks and reinforces the need for a sustainable, modular alternative for educational assignment management.

Beyond improving day-to-day assignment management, **RAW** establishes a unified workflow that bridges educational platforms and software development tools. Its modular architecture and open-source design enable future extensions, including support for Git hosting services beyond GitLab. By reducing platform fragmentation and standardizing practical assignment workflows, the project contributes to a more efficient, consistent, and scalable educational experience for both instructors and students.

Table of Contents

Preamble	I
Authentication	III
Abstract	V
1. Introduction	1
1.1. Motivation	1
1.2. Goal	1
2. Structure	2
3. Context	3
3.1. Moodle	3
3.2. GitLab	4
4. State of the art	6
4.1. GitLab Client	6
4.2. Integrations	6
4.2.1. GitHub Classroom	6
4.2.2. RepoBee	7
4.2.3. Classmoji	7
4.3. Related Research Project	8
5. Specifications	9
5.1. Objectives	9
5.2. Deliverables	9
5.3. Expected result	9
6. System architecture	10
6.1. Custom fields	10
6.2. Activity module	11
7. Features	13
7.1. Teachers	13
7.2. Students	14
7.3. Automation	15
7.4. Other	16
8. Planning and Timeline	17
8.1. Research and Specification	17
8.2. Workflow	17
8.3. Proof Of Concept	17
8.4. Reproducibility	18
8.5. System design	19
8.6. Implementation and Testing	20
8.7. Documentation and Thesis Writing	20
9. Development Methodology	21
9.1. GitLab client	21
9.2. Module creation	21
9.3. Custom fields	22
9.4. Database design	22
9.5. Teacher dashboard	23
9.6. Student view	24

9.7.	Automate GitLab workflows	26
9.8.	Bind Moodle user interface with GitLab	26
9.9.	Notifications, Calendar events and Tasks	27
9.10.	Webhooks	28
9.11.	Protected files	28
9.12.	Custom GitLab host	29
10.	Solution	30
10.1.	Overview	30
10.2.	Teacher Features	30
10.3.	Student Features	31
10.4.	Integration Flows	32
10.4.1.	Module creation	32
10.4.2.	Group creation	33
10.4.3.	Template repository webhook	34
10.4.4.	Group repository webhook	35
10.5.	Result	35
11.	Contributions, Limitations and Future Work	36
11.1.	Project Contributions	36
11.2.	Limitations	36
11.3.	Future Work	36
12.	Appendices	38
12.1.	Use of Artificial Intelligence	38
12.2.	Planning	38
12.3.	Weekly meeting reports	38
12.4.	Source code	38
12.5.	Project documentation	38
12.6.	Kanban Board	38
	Bibliography	39

List of figures

Figure 1	Moodle course	3
Figure 2	GitLab repository	4
Figure 3	Moodle plugin architecture	10
Figure 4	Course custom field for GitLab token	11
Figure 5	User custom field for GitLab username	11
Figure 6	new RAW activity module	12
Figure 7	RAW plugin interactions with GitLab	12
Figure 8	GitLab module on Moodle course page	13
Figure 9	Teacher dashboard scaffold	14
Figure 10	Student groups list scaffold	14
Figure 11	Student group scaffold	15
Figure 12	GitLab resources structure	16
Figure 13	POC GitLab module view	18
Figure 14	Resulting repository created on the selected GitLab group	18
Figure 15	laC architecture for RAW	19
Figure 16	Component interactions during group creation	20
Figure 17	Configure RAW activity module	22
Figure 18	Module plugin database schema	23
Figure 19	Teacher dashboard template repository	23
Figure 20	Teacher dashboard student groups	24
Figure 21	Different ways to retrieve group code	24
Figure 22	Student group selection interface	25
Figure 23	Student group interface	25
Figure 24	GitLab created group containing the template repository and a group repository	26
Figure 25	A Moodle submission calendar event	27
Figure 26	A Moodle submission reminder notification	27
Figure 27	GitLab template repository webhook	28
Figure 28	Plugin custom GitLab host configuration	29
Figure 29	Module creation flow	32
Figure 30	Group creation flow	33
Figure 31	Template repository webhook flow	34
Figure 32	Group repository webhook flow	35

1. Introduction

In higher education, digital platforms play a central role in course organization, the distribution of learning materials, and the management of academic activities. Learning Management Systems (LMS) such as Moodle are widely adopted to structure the educational experience for both teachers and students. At the same time, in computer science and related fields, source code management platforms like GitLab have become essential tools for distributing assignments, enabling collaboration, and evaluating practical work.

However, these two types of platforms, while complementary, operate independently. This separation leads to fragmented workflows: teachers must manually manage repositories, student groups, and assessment processes in GitLab while maintaining course-related information in Moodle. For students, this results in scattered resources and information across multiple platforms, leading to confusion and reducing the overall learning experience.

1.1. Motivation

The lack of integration between LMS platforms and Git repository hosting services introduces several challenges. Teachers often face repetitive and time-consuming tasks such as creating repositories for each student or group, managing access permissions, tracking contributions, and checking submission dates. These manual processes are not only inefficient but also error-prone, especially in courses with large enrollments.

Students, on the other hand, must navigate between multiple platforms to access learning materials, enrolment information, starter code, assessment results, and feedback. This fragmentation can create confusion and reduce engagement as important information is not centralized.

Existing solutions partially address these issues but remain limited. Some are proprietary and lack flexibility, while others are tightly coupled to specific platforms such as GitHub. Moreover, they are often not deeply integrated into LMS environments, requiring additional tools or workflows that increase complexity rather than reduce it. This situation highlights the need for an open, flexible, and LMS-native solution that seamlessly integrates Git-based workflows into the educational ecosystem.

Furthermore, GitHub Classroom, one of the most widely used platforms for managing computer science assignments, has announced that it will discontinue its service on 28 August 2026. This development highlights the risks of relying on proprietary, closed-source tools and reinforces the need for a new open-source, LMS-native solution.

1.2. Goal

To address these challenges, this project introduces **RAW** an extensible integration between Moodle and GitLab. The solution consists of a custom Moodle plugin that communicates with the GitLab API. This integration will enable direct interaction with GitLab features from within the Moodle interface.

The integration will automate workflows involved in programming assignments. This includes the creation and management of repositories, the automatic setup of student groups, the synchronization of project data, and the integration of feedback and grading mechanisms. By embedding these functionalities into Moodle, both teachers and students benefit from a unified platform where all relevant information and actions are accessible.

From an architectural perspective, **RAW** is designed to be modular and extensible. While initially focused on GitLab, the system will be structured to support future integration with other Git repository hosting services. This ensures long-term adaptability and encourages community contributions.

2. Structure

The remainder of this report is organized as follows:

- **Context** introduces the platforms and technologies involved in the project.
- **State of the Art** reviews and evaluates existing solutions related to the proposed integration.
- **Specifications** defines the project's objectives, deliverables, requirements, and expected outcomes.
- **System Architecture** describes the chosen architectural design.
- **Features** presents the initial requirements and expected functionalities of the integration, describing the objectives and capabilities targeted during development.
- **Development Process** outlines the main stages of the project's development and implementation, as well as the project management practices followed throughout its execution.
- **Solution** describes the final implemented integration, including the functionalities delivered and how the initial requirements were addressed.
- **Contributions, Limitations and Future Work** summarizes the project's contributions, presents potential improvements and directions for future development.
- **Appendices** contain the supplementary material accompanying this report.

3. Context

This chapter presents the main platforms and technologies underlying the proposed work. The presented information provides the necessary background to understand the role of each platform, their use in computer science education workflows, and their relevance to the proposed integration.

3.1. Moodle

Moodle [1] is an open-source Learning Management System (LMS) widely used by universities and educational institutions to organize courses, distribute learning materials, and manage academic activities. Its name stands for Modular Object-Oriented Dynamic Learning Environment. Moodle provides a modular structure based on courses, activities, and plugins, enabling institutions to adapt the platform according to their specific requirements.

While working and studying at Curtin University, Martin Dougiamas, the creator of Moodle, gained experience with the WebCT LMS (now discontinued and replaced by Blackboard products [2]), which led him to investigate an alternative approach to online teaching. In 1999, he started developing and testing early prototypes of a new LMS. The first version of Moodle was released in 2002, and the platform gradually grew into one of the most widely adopted learning management systems worldwide.

“Now, nearly 450 million educators and learners worldwide use Moodle’s adaptable platform.” [3]

Finally, its large global community and long-term development have contributed to its reliability, customization, and sustainability.



Figure 1: Moodle course

In many institutions, Moodle acts as a central platform for teaching activities, including document sharing, assessment management, quizzes, and evaluation. For example, platforms like Cyberlearn

(used within HES-SO) are built on Moodle and extend its capabilities to meet institutional requirements.

As stated in the official Moodle [documentation](#) [4], one of Moodle’s main strengths lies in its modular architecture.

“There are other types of software systems that are important for educational institutions [...] Generally, Moodle does not try to re-invent these areas of functionality. Instead, it tries to be the best LMS possible, and then interoperate gracefully with other systems that provide the other areas of functionality.”

This extensible architecture has led to a large ecosystem of plugins and integrations, allowing Moodle to be customized for various use cases. However, implementing advanced workflows often requires custom development tailored to the platform’s architecture.

A well-known alternative to Moodle is [Canvas](#) [5]. While Moodle focuses on extensibility and customization, Canvas primarily emphasizes ease of use and straightforward setup.

The Moodle ecosystem is primarily developed using PHP [6], with JavaScript used for client-side functionality. Therefore, the developed Moodle plugin must follow the same technological stack and comply with the development guidelines defined by the Moodle platform.

3.2. GitLab

[GitLab](#) [7] is an open-source Git repository management platform that provides tools for source code and project management as well as continuous integration (CI) / continuous deployment (CD).

The main GitLab features include source code hosting and management via Git repositories, merge requests and code review workflows, issue tracking and project management tools, and CI/CD pipelines for automated testing and deployment. Additionally, GitLab provides extensibility through integrations that use Webhooks and REST/GraphQL APIs.

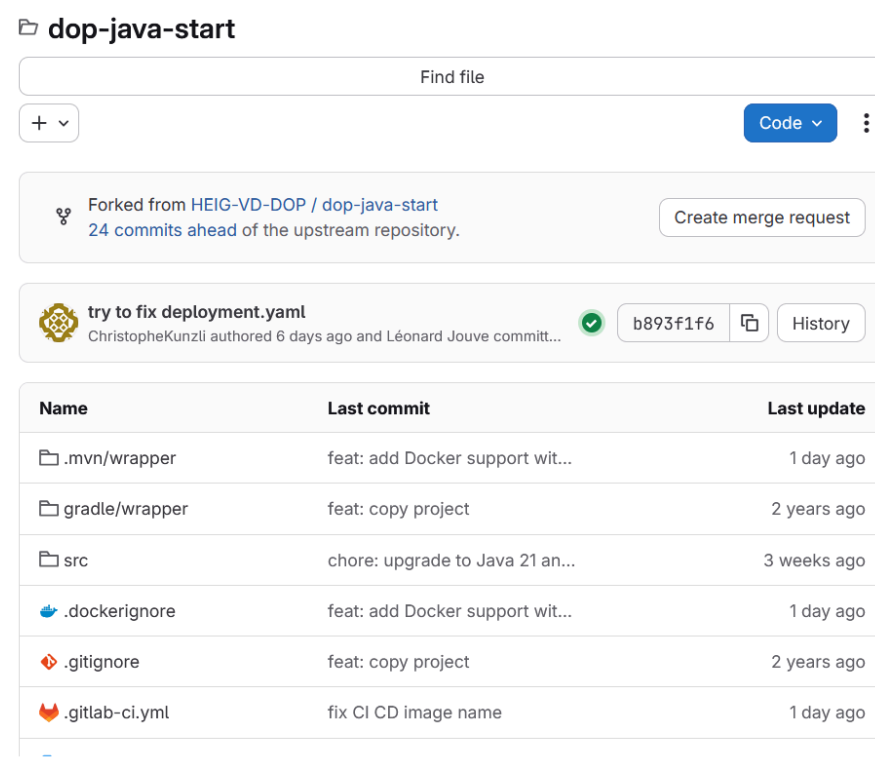


Figure 2: GitLab repository

GitLab is widely used in both industry and education due to its full DevOps lifecycle integration and strong automation capabilities. In academic contexts, it is commonly used for programming assignments, enabling students to collaborate and submit code while allowing teachers to review and evaluate contributions efficiently.

Its open-source nature allows GitLab to be self-hosted and customized. This is particularly relevant in education and for large companies, where organizations may have strict requirements regarding data privacy, infrastructure control, and long-term availability. This ability to adapt GitLab to different organizational needs has been a key factor in its success and in its adoption as a widely used software development platform.

Another option is GitHub, a Git hosting platform that offers features similar to GitLab. However, GitHub follows a different approach, relying more heavily on its developer community and ecosystem of external integrations. At the same time, GitLab provides a more unified platform for managing the software development lifecycle.

4. State of the art

The project is divided into two main phases. The first phase focuses on implementing interactions with GitLab through its REST API, while the second phase consists of developing a Moodle plugin to provide the user interface and integrate the solution within the Moodle ecosystem. These two components must be combined to deliver a complete and functional integration.

4.1. GitLab Client

On the GitLab API side, there are multiple existing GitLab Client solutions. An HTTP Client is an abstraction layer wrapping REST API endpoints behind a user-friendly interface.

Several implementations exist across different programming languages and ecosystems:

- [GitLab4J](#) [8] provides a full-featured, easy-to-use Java library for working with the GitLab REST API, making it suitable for building Java-based applications that interact with GitLab.
- [GitLab Haskell](#) [9] is a Haskell library developed as part of the [Canvas GitLab Integration](#) [10] research project, which focuses on integrating GitLab with the Canvas learning management system. This project represents a similar academic effort to our work and will be discussed in more detail in a later section.
- [python-gitlab](#) [11] is a Python package widely used for automation and scripting tasks involving GitLab.

Since the Moodle plugin is developed in PHP, [GitLab PHP API Client](#) [12] seems to be the most suitable existing implementation. It provides support for the full GitLab API v4 specification.

However, Moodle plugins are designed to be self-contained and should avoid introducing unnecessary external dependencies. For this reason, a dedicated GitLab client will be implemented as part of this project, while taking inspiration from existing solutions and following their established design principles.

4.2. Integrations

Several existing solutions aim to bridge Learning Management Systems (LMS) with Git repository hosting platforms. These integrations mainly focus on simplifying programming assignment management, automating repository creation, and connecting educational workflows with software development tools.

4.2.1. *GitHub Classroom*

[GitHub Classroom](#) is an assignment management platform integrated with GitHub.

This solution automates the creation and setup of student repositories for teachers.

Teachers can specify an assignment name, due date, group size, starter code repository, CI jobs to run, a list of protected files to monitor for unauthorized modifications, and a feedback pull request workflow. Changes to the template repository can be synchronized across all student repositories through pull requests. Reviewers are also notified when submissions are late.

GitHub Classroom is currently one of the most widely used solutions for computer science education assignments. However, its main limitation is that it is not directly integrated with the school's LMS. Instead, it introduces an additional layer of complexity between the LMS and the Git hosting platform.

Another drawback is that it integrates only with GitHub, a closed-source platform. This can lead to vendor lock-in.

This solution will heavily inspire the workflows that teachers will be able to automate with our plugin, and its main features will be available.

On 22 May 2026, GitHub Classroom published a public [announcement](#) [13] of the end of their service for 28 August 2026:

“For many educators using GitHub Classroom, you may know that this product has been operating under ‘maintenance mode’ for the past 18 months. We know this has been frustrating, and we want to make sure Classroom gets the investment it deserves.”

Following this decision, GitHub redirected users toward partner solutions:

- [Codio](#) [14]: A learning platform providing browser-based coding environments, automated grading capabilities, and LMS integration features. However, its LMS integration remains limited, and the platform is a closed-source premium solution.
- [Classroom 50](#) [15]: A free and open-source assignment management platform developed by the Fifty Foundation. It enables instructors to distribute programming assignments, configure automated grading workflows via GitHub Actions, and manage student submissions through both web and command-line interfaces. This solution appears promising. However, it was only released on 1 July 2026, one week before this writing, and does not currently integrate directly with LMS platforms.

4.2.2. RepoBee

[RepoBee](#) [16] is a command-line tool that enables teachers to manage large numbers of student Git repositories across multiple version control systems. It features a powerful plugin system that allows users to either use existing plugins or develop their own. However, this solution does not integrate directly with any LMS platform.

4.2.3. Classmoji

[Classmoji](#) [17] is an open-source LMS for teaching computer science. It is not widely adopted yet, but it is built with GitHub integration in the core. It claims to integrate GitHub Classroom directly with the LMS and add additional features.

	Open source	LMS integration	VCS independent	Extensible
GitHub Classroom	~	×	×	×
Codio	×	~	×	×
Classroom 50	✓	×	×	~
RepoBee	✓	×	✓	✓
Classmoji	✓	✓	~	~
This work	✓	✓	~	✓

✓ Supported, ~ Partially supported, × Not supported.

Comparison of existing solutions

As shown in the comparison, existing solutions either lack deep LMS integration, rely on proprietary platforms, or are limited to GitHub. Currently, there is no open-source, Moodle-native integration with GitLab that supports automated assignment workflows. This gap motivated the development of **RAW**, which aims to provide an extensible, open-source solution for managing assignments through Version Control Systems directly within the LMS platform.

4.3. Related Research Project

Canvas GitLab Integration [10] is a research project aiming to enrich learning processes.

The paper highlights that separating version control systems (VCS) from learning management systems (LMS) leads to a fragmented and inefficient user experience, as students and instructors must switch between platforms. To address this, the authors propose a high-level programming framework that integrates Canvas and GitLab to reduce fragmentation and improve the overall educational experience.

It is a project in 3 parts:

- GitLab API
- Moodle API
- Moodle - GitLab integration API

All 3 projects are open source and written in Haskell.

The study investigated the experiences of Computer Science educators and students using Canvas and GitLab through a survey and identified challenges stemming from the lack of integration between the two platforms. The results showed that educators faced significant administrative workloads, including manually tracking student activity, linking GitLab projects with coursework, and monitoring student engagement. Students also reported difficulties in associating coursework on Canvas with their GitLab projects and frequently switching between the two systems.

This report also mentions that a key challenge in designing automated systems is balancing automation with flexibility. While automation can reduce repetitive tasks, improve efficiency, and minimize human error, excessive automation may limit users' ability to adapt workflows to specific needs or unexpected situations. Rigid automated processes can create frustration when users require control or alternative approaches. Systems that provide too much flexibility, on the other hand, may reduce the benefits of automation by increasing complexity and requiring additional manual effort. Therefore, effective system design should aim for an appropriate balance, automating labor-intensive tasks while preserving sufficient flexibility to allow users to make decisions and adjust processes.

5. Specifications

5.1. Objectives

This project aims to:

- Review the state of the art and identify the features and functionalities the project will include based on possibilities and limitations within the GitLab and Moodle APIs.
- Evaluate and identify the most suitable architecture for the implementation.
- The codebase should be modular, open to modifications and improvements.
- Implement, develop, and deploy the integration between Moodle and GitLab by following DevOps principles.
- Validate and test (optionally with representative users).

5.2. Deliverables

By the end of this work, we aim to have the following deliverables:

- Work report
 - Specifications document
 - Gantt chart
 - State of the art
- Work presentation
 - PowerPoint
 - Poster
- Plugin code published as open source
 - Usage documentation
 - Code tests
 - System architectural design
 - Demonstration of a working prototype

5.3. Expected result

The goal of this project is to implement an extensible integration between GitLab (and potentially other Git repositories) and Moodle. It would include an API that automates all the mentioned issues, accessible via a Moodle integration plugin, thereby improving teachers' and students' workflows.

The setup and use of the plugin should be easy and intuitive.

It should help teachers as well as students with their organization, document/information centralization, and the automation/simplification of tedious workflows.

It should be released as open source so others can access, use, and build upon the work.

6. System architecture

As mentioned previously, one of Moodle's main strengths is its modularity and extensibility. It was designed around a comprehensive plugin architecture, allowing developers to extend its functionality through a wide range of plugin types. The appropriate plugin type should be selected based on the specific requirements of the desired functionality.

Some of the available plugin types include:

- **Activity modules** – Provide learning activities within courses (e.g., Forum, Quiz, Assignment).
- **Custom field** – Define custom field types used in course/profile custom fields.
- **Web service** – Implement new protocols for web service communication.
- **Assignment submission** – Provide additional methods for students to submit assignments.

For this project, the development focused on two plugin types:

- **Activity modules**
- **Custom field**

Plugins must be placed in a directory corresponding to their plugin type. The Moodle core automatically detects and registers these plugins during installation or upgrade. Once registered, their code is loaded and executed whenever the associated functionality is required.

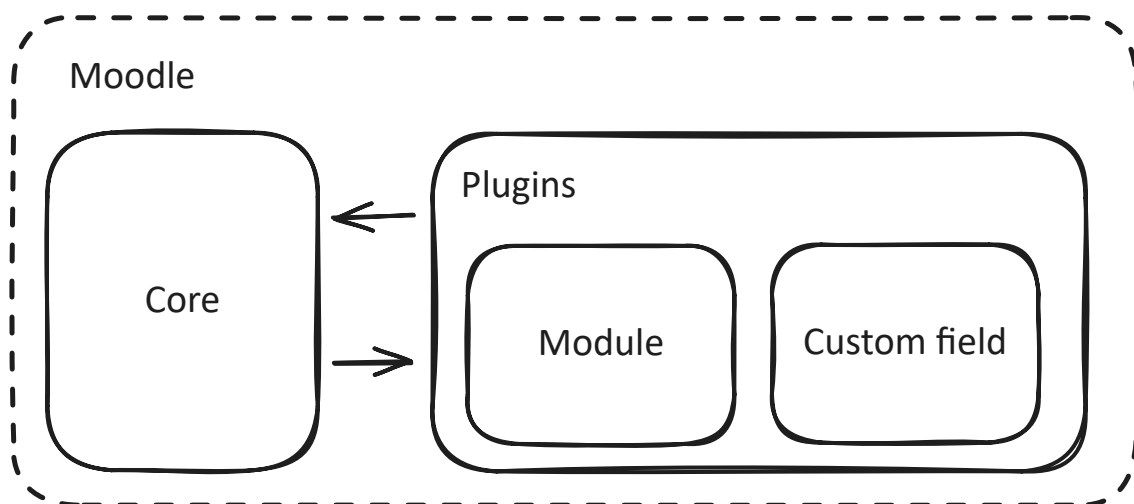


Figure 3: Moodle plugin architecture

6.1. Custom fields

The developed **Custom field** plugin adds a new custom field type for storing sensitive content. The field data is encrypted using the Moodle [built-in encryption](#) [18] mechanism before being stored in the database, and is decrypted when accessed.

Custom fields are additional pieces of information that can be configured and filled in through the Moodle interface. They can be added at different levels:

- **User** level
- **Course** level
- **Group** level

Each custom field can store different types of data, including:

- **Boolean** values

- **Text**
- **Integer** values
- The newly introduced **encrypted text** type

The plugin defines two custom fields:

- A **course-level** field for storing a course-specific GitLab token.
- A **user-level** field for storing the user's GitLab username.

devops

The screenshot shows two Moodle pages. The left page is 'Edit course settings' with tabs for Course, Settings, Participants, and More. The 'Settings' tab is active, showing a list of settings categories: General, Description, Course format, Appearance, Files and uploads, Completion tracking, Groups, Tags, and GitLab. The 'GitLab' category is expanded, showing a 'GitLab Token' field with a masked input and a 'Save and display' button. The right page is 'Edit profile' for 'Admin User'. It shows a list of profile categories: General, User picture, Additional names, Interests, Optional, and GitLab. The 'GitLab' category is expanded, showing a 'GitLab Username' field with the value 'LeonardJouve' and 'Update profile' and 'Cancel' buttons.

Figure 4: Course custom field for GitLab token

The GitLab token field uses the newly created encrypted text field type to encrypt the token before it is stored in the database. This ensures that sensitive credentials are not stored in plaintext and cannot be directly accessed through database inspection.

The built-in Moodle encryption system is implemented using the PHP Sodium extension, which provides the XSalsa20 stream cipher for encryption and the Poly1305 message authentication code (MAC) for integrity verification. The encryption key is stored in a file external to the database, preventing exposure of the key in the event of a database compromise.

6.2. Activity module

The **Activity module** plugin introduces a new Moodle activity type called **RAW**. This module is the central component of the integration between Moodle and GitLab. Creating a **RAW** activity requires a GitLab token defined at the course level, as well as a GitLab username for each user who accesses the activity.

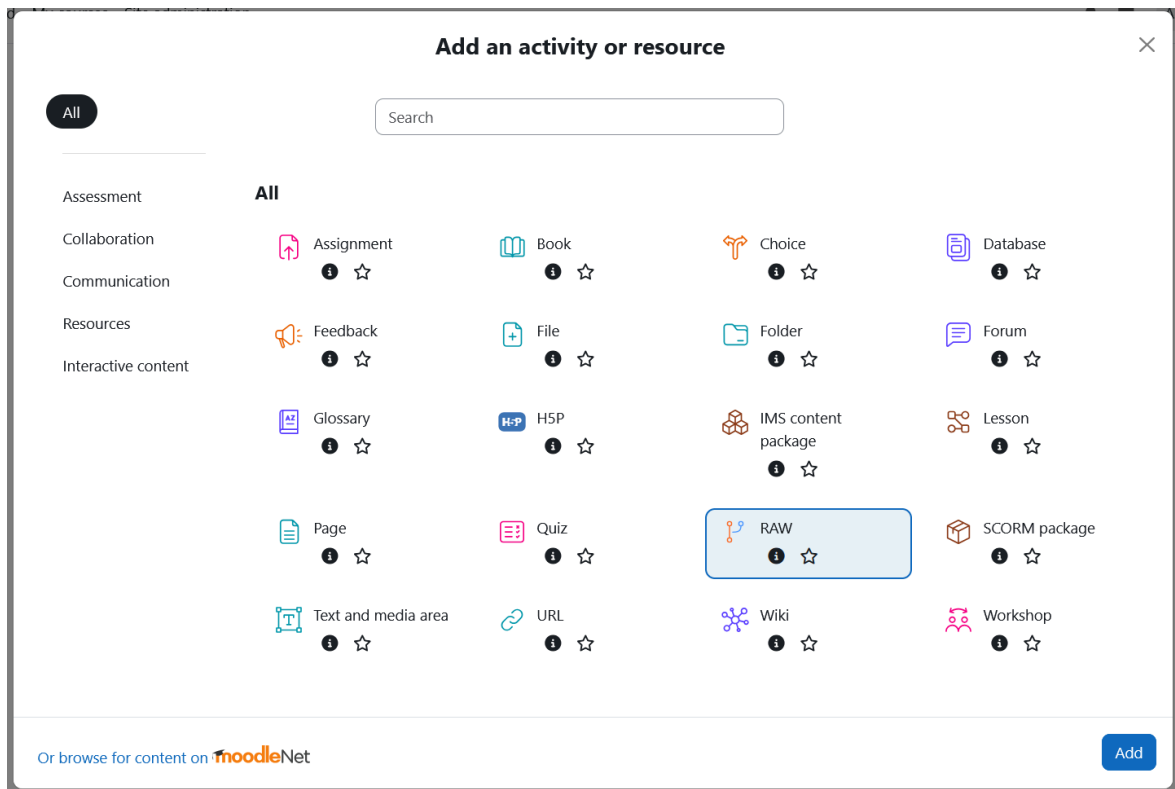


Figure 6: new **RAW** activity module

The plugin acts as the primary bridge between Moodle and GitLab. Unlike GitHub Classroom, which provides a standalone platform separate from learning management systems, **RAW** offers a fully integrated solution within Moodle, allowing Git-based assignment workflows to be managed directly from the LMS.

RAW communicates with Moodle Core to retrieve course and user data, create activity instances, and manage Moodle-side information. A dedicated GitLab Client component handles all communication with GitLab.

The GitLab Client interacts with the GitLab REST API to provision and manage GitLab resources according to the desired state. Using the REST API keeps both platforms decoupled: Moodle does not need to know about GitLab's internal implementation, and GitLab does not require any Moodle-specific configuration.

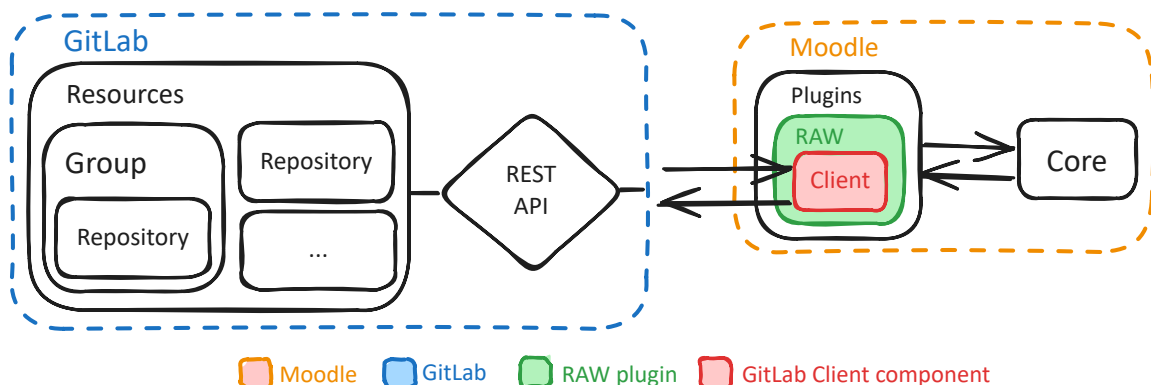


Figure 7: RAW plugin interactions with GitLab

7. Features

The following features represent the initial requirements identified for the integration. These requirements define the expected behavior and scope of the solution, including the management of GitLab resources, the interaction between Moodle and GitLab, and the handling of user and course-specific data.

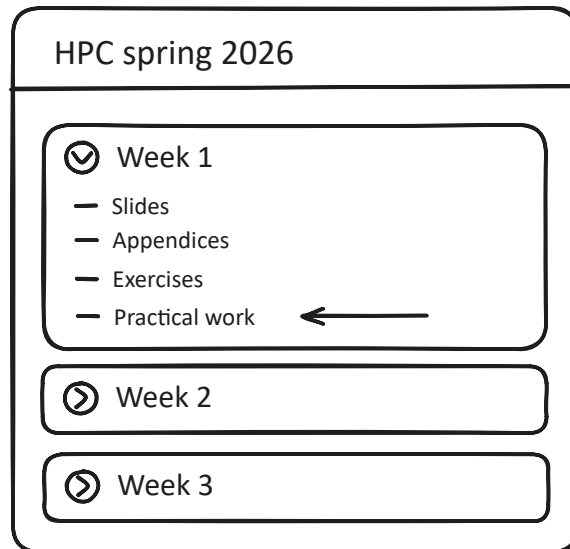


Figure 8: GitLab module on Moodle course page

7.1. Teachers

As a teacher, I want to:

- *define practical work properties such as name, description, instructions, due date, group size, reviewers*
- *create a RAW activity module in Moodle so that I can manage a practical work linked to GitLab repositories.*
- *access a GitLab template repository so that I can prepare the initial project and solution.*
- *view all student groups so that I can monitor participation and project progress.*
- *create, edit, or remove student groups so that I can organize collaborative work.*
- *access each group repository so that I can review submissions efficiently.*
- *see the latest CI/CD pipeline results so that I can quickly identify failing projects.*
- *view individual contribution statistics so that I can assess each student's participation.*
- *download repository source code so that I can review projects locally.*
- *know whether a group has already been graded so that I can avoid duplicate corrections.*

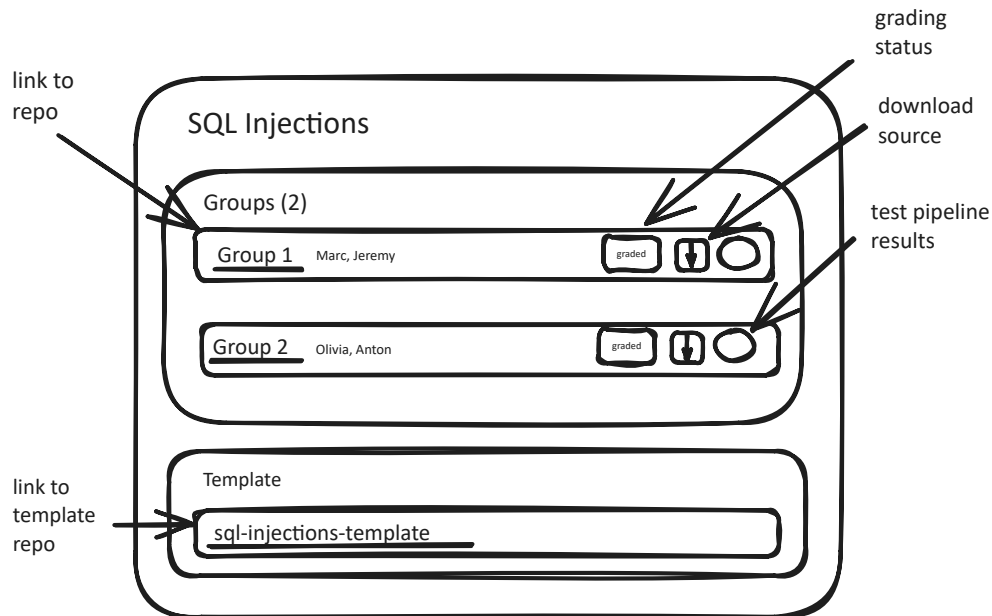


Figure 9: Teacher dashboard scaffold

7.2. Students

As a student, I want to:

- view the list of existing groups so that I can join a team.
- join a group so that I can collaborate with other students.
- create a new group so that I can start a project team.
- access my group repository so that I can contribute to the practical work.
- view grades and feedback on Moodle and GitLab merge requests so that I can understand my evaluation.
- receive a notification when grading is completed so that I know when feedback is available.
- see the assignment due date in the Moodle calendar so that I can manage my schedule.

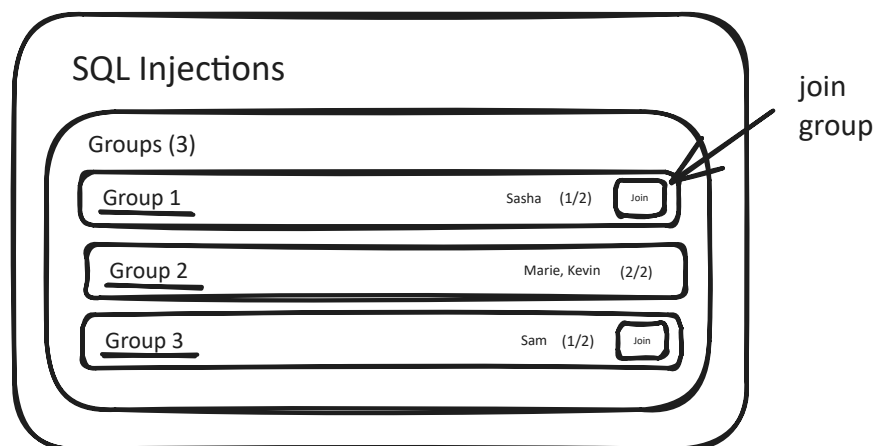


Figure 10: Student groups list scaffold

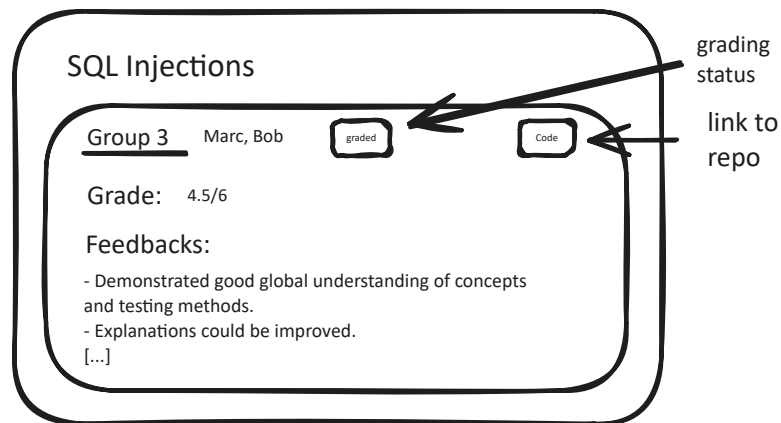


Figure 11: Student group scaffold

7.3. Automation

As a teacher, I want to:

- *create a GitLab group automatically when a Moodle activity is created so that setup is simplified.*
- *create a template repository automatically so that all student repositories share the same base project.*
- *see updates to the template repository propagate to student repositories through merge requests so that projects stay synchronized.*

As a student, I want to:

- *have a repository created automatically when my group is formed so that I can start working immediately.*
- *see the practical work instructions automatically added as GitLab issues so that I can easily track requirements.*
- *have a solution branch become available after the deadline so that I can compare my work with the expected solution.*

Gitlab resources

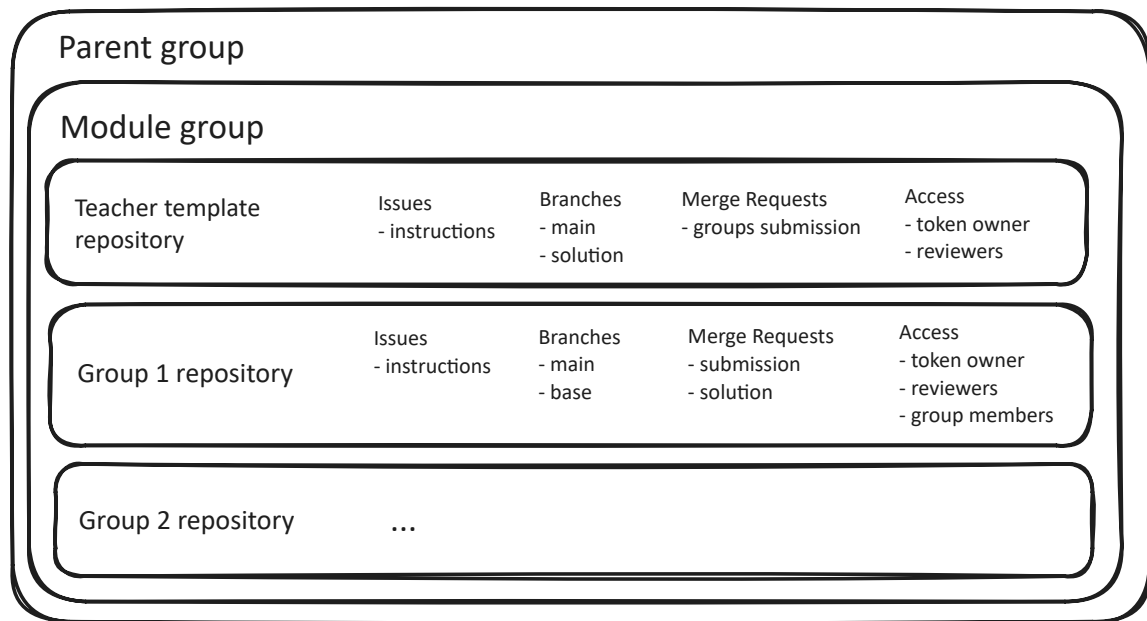


Figure 12: GitLab resources structure

7.4. Other

Some custom integrations could be implemented with external tools, but these have been defined as out of scope for this project and could be added later through separate Moodle plugins.

Potential candidates include assessment platforms such as [Gradescope](#) [19], and [ANS](#) [20]. However, integrating with these proprietary platforms would introduce dependencies on third-party services and may not suit all teaching workflows, as instructors often have different preferences.

8. Planning and Timeline

This project was carried out over the course of one academic semester, with a total workload of 450 hours for the various activities required to complete it.

8.1. Research and Specification

The first months were mainly focused on establishing the project requirements, defining the scope, and identifying feasible features. This phase also included a state-of-the-art analysis to examine existing solutions, evaluate current approaches, and identify the limitations and gaps that this project addresses. The analysis provided a better understanding of the existing ecosystem and helped guide the design choices.

The outcome of this phase was a well-defined feature list, a Gantt chart Section 12.2, and a Kanban board Section 12.6, providing a structured roadmap for the project development.

8.2. Workflow

The development workflow for this project used a Git feature-branch strategy to ensure isolated, organized, and traceable changes. Each feature was implemented in a separate Git branch, allowing for review before integration into the main codebase. Task management was handled via a [Kanban board](#) [21]. Items were organized by priority and annotated with their dependencies. This ensured critical tasks were addressed first and provided visibility into blocking/critical tasks.

8.3. Proof Of Concept

The next step was to build a proof of concept [22], a simple GitLab integration within Moodle. This included creation of a Moodle module plugin. This kind of plugin provided a new type of resource a teacher can add to a course.

The teacher must provide a GitLab token when creating the resource. In the final **RAW** plugin, this token was then transferred to the course level.

Once created, students can open the module and create a GitLab repository. They can also see a list of existing repositories in the GitLab group, with direct links to each.



Figure 13: POC **GitLab** module view



Figure 14: Resulting repository created on the selected GitLab group

8.4. Reproducibility

To provide a reproducible and automated deployment environment, an Infrastructure-As-Code approach was adopted using [Terraform](#) [23]. Terraform uses a declarative configuration model, allowing the desired state of the infrastructure to be described rather than specifying the sequence of operations required to achieve it. It was used to define and provision the required cloud infrastructure, including creating and configuring an AWS EC2 instance with an Elastic IP address.

Once the infrastructure was provisioned, Configuration-as-Code principles were applied using [Ansible](#) [24]. Ansible playbooks were created to automate the installation and configuration of the software environment. This approach ensures that the deployment process is repeatable and maintainable.

The deployed environment consists of a Moodle instance protected by [Authentik](#) [25] as an identity provider. Authentik was selected because it provides an open-source authentication solution that supports modern authentication protocols with minimal configuration, reducing complexity.

To handle communication between the different services, [Traefik](#) [26] was used as a reverse proxy and ingress controller. Traefik is particularly well-suited for containerized environments due to its ease of use and its ability to dynamically configure routing rules based on running services.

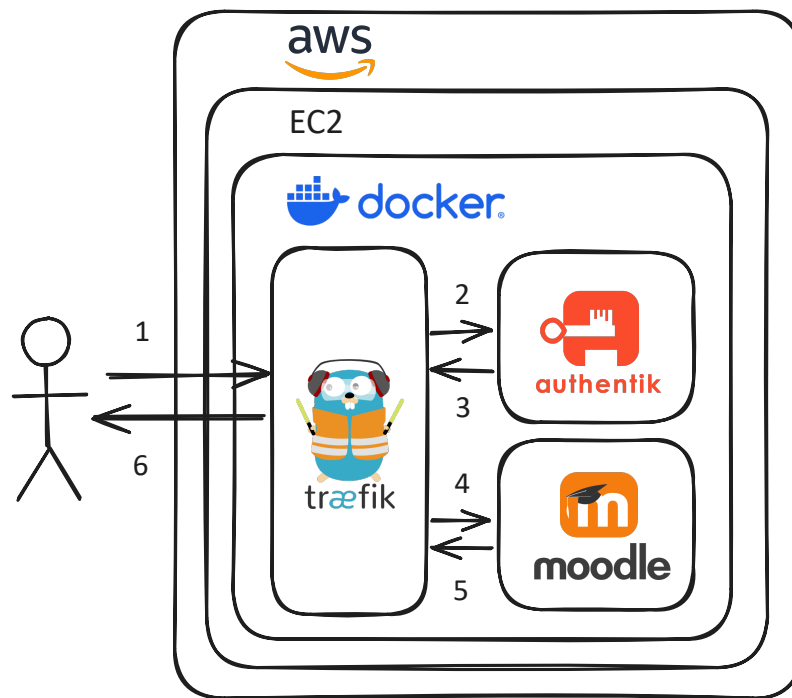


Figure 15: IaC architecture for RAW

Finally, a GitHub Action was added to automate updates after each push to the project's main branch.

8.5. System design

The solution was designed around a modular architecture separating Moodle-specific logic, GitLab resource management, and API communication. It follows the Moodle Activity module plugin structure and naming conventions described in the official documentation.

The main design goals were to keep Moodle and GitLab independent by centralizing GitLab communication within a dedicated client component, and, as much as possible, follow the design principles and coding practices used in the Moodle codebase.

The plugin is divided into multiple components:

- The **Bridge** component coordinates between Moodle and GitLab and manages the creation of required resources on both platforms.
- The **Resources** component manages the creation and configuration of GitLab resources.
- The **GitLab Client** provides a simplified interface over the GitLab REST API endpoints.

The Bridge component delegates GitLab-related operations to the Resources component and updates the corresponding Moodle database records to maintain synchronization between both systems.

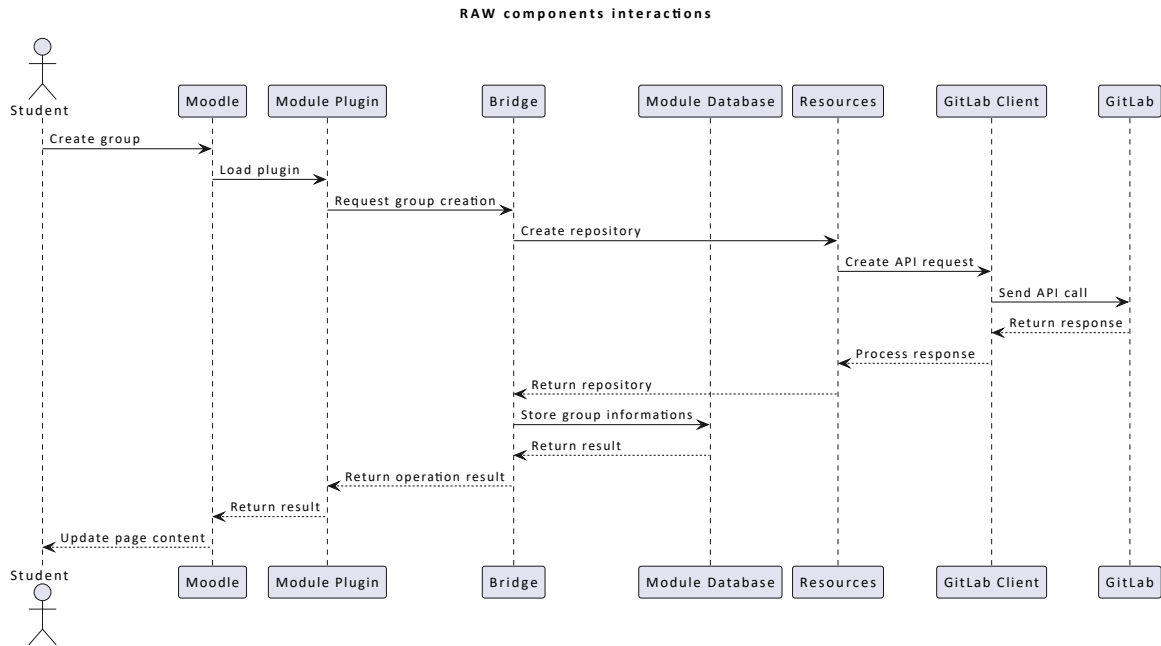


Figure 16: Component interactions during group creation

8.6. Implementation and Testing

This phase covered the development of Moodle plugins and the combination of their components. It included implementing the GitLab client, Moodle user interfaces, automated workflows, and required features. Unit tests were developed alongside the implementation to validate individual components and ensure the correctness of the functionalities. Additional manual testing was performed to identify issues, verify integrations, and improve the solution's overall reliability.

8.7. Documentation and Thesis Writing

The final phase focused on documenting the developed solution and writing the thesis report. It included preparing technical documentation, describing the architecture and implementation choices, presenting the results, and summarizing the lessons learned throughout the project. The report was continuously updated throughout the project to reflect the implementation's evolution and maintain accurate documentation.

9. Development Methodology

This chapter presents the main stages of the development process and describes the progression of the work throughout the project.

9.1. GitLab client

The GitLab client component provides an abstraction layer over the GitLab REST API. It centralizes all communication with GitLab and exposes high-level methods for interacting with GitLab resources.

This abstraction isolates GitLab-specific implementation details from the rest of the plugin. It simplifies resource management, improves maintainability, and allows future changes to the GitLab API communication layer to be handled independently from the Moodle integration logic.

The client manages authentication by attaching the given GitLab token to each request and provides access to dedicated components responsible for different GitLab resource types.

The design of the GitLab client was inspired by the structure and usage patterns of the previously introduced [GitLab PHP API Client](#) [12] library.

The `Gitlab` class acts as the main entry point and exposes dedicated objects for different GitLab resource types:

- `group()` handles GitLab groups
- `project()` handles repositories/projects
- `user()` handles users
- `branch()` handles branches
- `member()` handles permissions and memberships
- `issue()` handles issues
- `merge_request()` handles merge requests
- `pipeline()` handles CI/CD pipelines
- `webhook()` handles webhooks
- `file()` handles repository files

This means that other parts of the Moodle plugin can write code like:

```
$client->group()->create($name, $parent_id, $extra);  
$client->project()->fork($repository_id, $name, $group_id, $extra);
```

9.2. Module creation

The activity module implementation introduces the **RAW** activity type into Moodle. The module stores the configuration required for each activity instance, including the name, description, selected GitLab parent group, maximum group size, reviewers, and submission due date.

New RAW

[Expand all](#)

General

Name

!

?

Kubernetes

Description

Edit View Insert Format Tools Table Help

↶ ↷ B I       

Objectives

- Access the course Kubernetes cluster.
- Deploy an application to Kubernetes.
- Create a `deployment.yaml` file to deploy the application.

div > section > div > div > div > div > div > div > ul > li 19 words Build with  tinyMCE

Parent group

!

?

kubernetes

Group size

!

?

2

Assignment reviewer

No selection

Search

▼

Due date

!

?

27

July

2026

17

03



Figure 17: Configure **RAW** activity module

9.3. Custom fields

Custom fields were implemented to store additional information required by the integration within the appropriate Moodle contexts.

Two custom fields were introduced:

- A course-level field storing the GitLab token used to manage resources for a specific course.
- A user-level field storing the GitLab username associated with a Moodle user.

The integration links Moodle users with their GitLab identities through the configured GitLab username field.

This association allows the plugin to assign GitLab permissions when creating repositories and groups automatically.

Repository permissions are configured according to user roles:

- Students receive access to their group repository with the **developer** access level.
- Teachers and reviewers receive the **maintainer** access level.

9.4. Database design

The plugin uses dedicated database tables to store the relationships between Moodle activities and GitLab resources.

RAW: a GitLab Integration within Moodle

22

21 July 2026

The following tables contain the data required for the plugin operation:

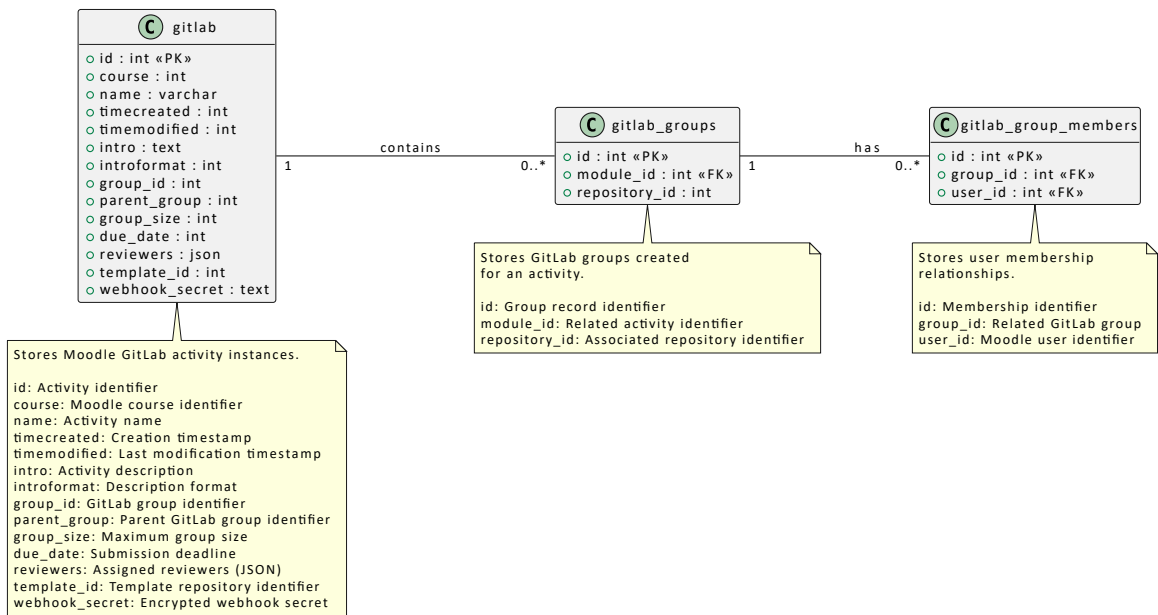


Figure 18: Module plugin database schema

9.5. Teacher dashboard

The teacher interface provides access to the main activity settings, including the module name and description, the GitLab parent group where resources are created, the assigned reviewers, and the submission due date. All deadlines are interpreted using the configured Moodle instance's time zone to ensure consistency.

Teachers and reviewers have direct access to the GitLab template repository associated with the activity. This repository serves as a central workspace for preparing practical work, defining the initial project structure, instructions, and required resources before group repositories are created.

Template

Kubernetes_template

Reviewers

Due date

Repository

Instructions

Solution

Release solution

Get source code

Monday, 27 July 2026, 5:03 PM

Open

Open

Open

Release

Code ▾

Figure 19: Teacher dashboard template repository

The interface also provides an overview of the student groups associated with the activity. Teachers can create and manage groups, assign members, and access the corresponding GitLab repositories and submission information.

Template

> Kubernetes_template

Groups

Group of user1/2

Membersuser

Repository

FeedbacksNot graded

Get source codeOn time

Delete group

Edit

Open

Open

Code

Delete

Create

Clone with SSHgit@gitlab.com:practic

Clone with HTTPShttps://gitlab.com/prac

Checkout version at due dategit checkout 84232f0e

Checkout from templategit fetch origin merge-
git checkout mr-1

Download source codeAt due dateLatest

Code

Figure 21: Different ways to retrieve group code

Figure 20: Teacher dashboard student groups

For evaluation purposes, reviewers and teachers can retrieve group submissions using different methods. They can clone repositories using SSH or HTTPS, inspect the exact repository state at the submission deadline by checking out the corresponding commit, review submissions through merge requests centralized in the template repository, or download the source code as a ZIP archive.

9.6. Student view

The student interface provides access to GitLab-related information directly from Moodle.

Students who are not assigned to a group can view available groups, create a new group, or join an existing group that has not reached its maximum capacity.

RAW: a GitLab Integration within Moodle

24

21 July 2026

Kubernetes

Objectives

- Access the course Kubernetes cluster.
- Deploy an application to Kubernetes.
- Create a `deployment.yaml` file to deploy the application.

Groups

Empty group

0/2 [Join](#)

[Create](#)

Figure 22: Student group selection interface

Once a student becomes a member of a group, additional options become available, including access to the group's GitLab repository and submission-related information.

The integration allows students to follow the complete submission workflow in Moodle without manually switching between Moodle and GitLab.

Kubernetes

Group

Group of user

1/2

Members

user

Repository

[Open](#)

Feedbacks

Not Graded

[Open](#)

Solution

[Open](#)

Get source code

On time

[Code](#) 

Leave group

[Leave](#)

Figure 23: Student group interface

The interface also provides information on each group's submission status. Students can identify whether their work has been graded, submitted on time, or submitted after the deadline.

After the submission deadline, students can also access the solution provided by teachers directly from the Moodle interface.

9.7. Automate GitLab workflows

The plugin automates the creation and management of GitLab resources required for practical assignments.

For each activity, the plugin automatically creates a dedicated GitLab group inside the selected parent group. This group acts as a container for all resources associated with the Moodle activity.

A template repository is also created for the practical work. This repository is accessible to the GitLab token owner and the selected reviewers, and it provides the initial project structure used to generate student repositories. It contains the practical work instructions as an issue, which is replicated to each group repository, as well as two branches: the `main` branch used as the template for initializing group repositories and the `solution` branch containing the reference solution. The solution branch can be released to each group through a merge request at any time by teachers.

For each student group, the plugin creates a fork of the template repository and configures access permissions according to the user's role. Group members receive Developer access, the GitLab token owner receives Owner access, and selected reviewers receive Maintainer access.

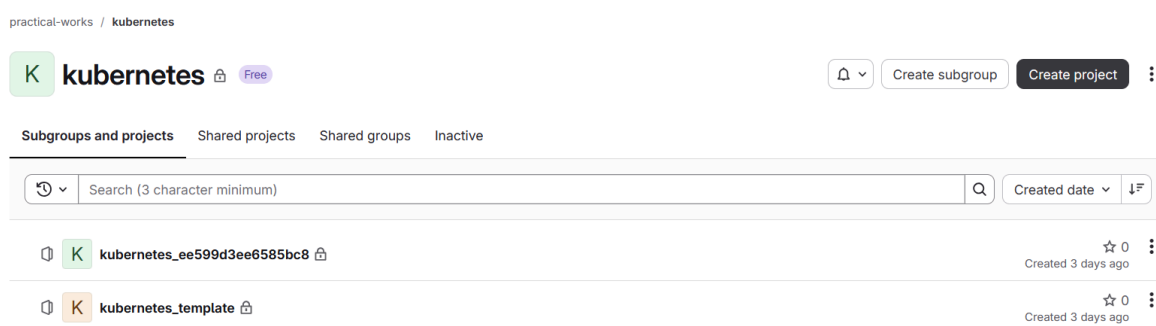


Figure 24: GitLab created group containing the template repository and a group repository

Each group repository contains a copy of the practical work instructions issue and two branches: the `main` branch, used by students to submit their work, and the `base` branch, used as a protected baseline for tracking modifications. A merge request from `main` to `base` is automatically created to simplify the review process.

After the submission deadline, each group submission merge request is also mirrored to the template repository, allowing reviewers to access all submissions from a single location.

This automation removes repetitive manual configuration, ensures consistency across assignments, and provides a standardized workflow for managing practical work.

9.8. Bind Moodle user interface with GitLab

In the previous steps, links between Moodle and GitLab resources were represented using placeholders. The goal of this step was to replace these placeholder values with the actual links corresponding to the GitLab resources created during the workflow.

9.9. Notifications, Calendar events and Tasks

The plugin integrates with Moodle notification and calendar systems to provide deadline reminders and keep users informed about relevant events.

Submission due dates are added to the Moodle calendar, allowing students to track upcoming deadlines directly from the Moodle interface.

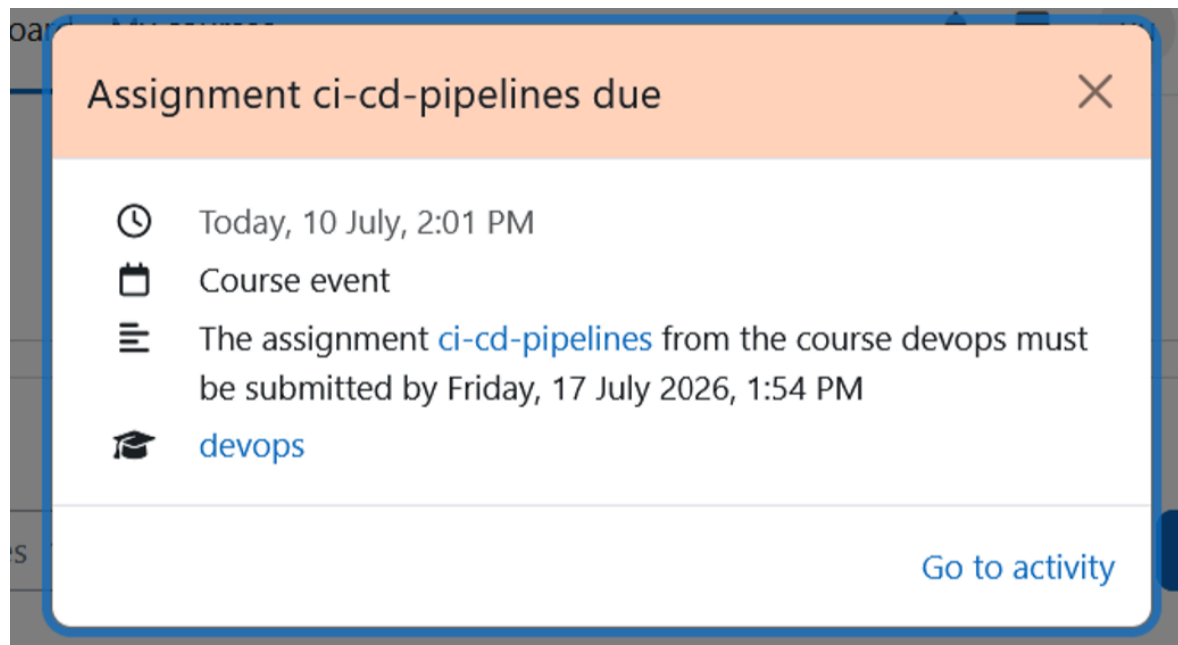


Figure 25: A Moodle submission calendar event

Notifications are generated for different events, including:

- One day before the submission deadline.
- The submission deadline date.
- Submission evaluation completion after the submission merge request is closed.

Notifications

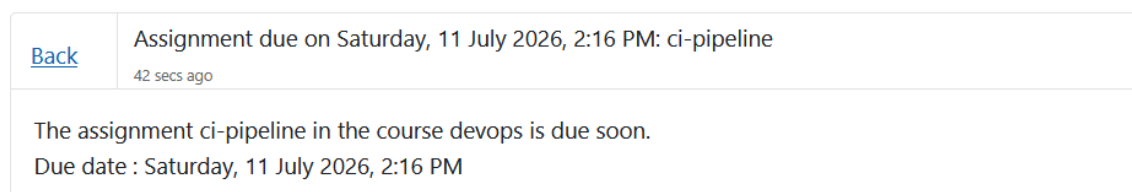


Figure 26: A Moodle submission reminder notification

The plugin also uses Moodle Core's Adhoc Tasks mechanism to execute asynchronous operations. This allows developers to schedule tasks for later execution while Moodle manages their execution through the `cron.php` script.

The `cron.php` script is a required component of any Moodle instance and must be executed regularly using a system cron job, as described in the official [documentation](#) [27].

9.10. Webhooks

Webhooks were implemented to synchronize GitLab events with Moodle and maintain consistency between the two platforms. They are automatically created and configured in the repositories provisioned by the plugin.

GitLab sends signed webhook requests to the Moodle endpoint:

```
https://<moodle-host>/mod/gitlab/webhook.php
```

To ensure the authenticity and integrity of webhook requests, GitLab signs each payload using a module-specific secret generated by Moodle. This secret is stored encrypted in the Moodle database using the same encryption method as for the GitLab tokens. The plugin uses it to verify each incoming webhook request before processing the event.

The webhook endpoint must be publicly accessible without any authentication layer on top, as requests are sent directly by GitLab. If authentication cannot be disabled, additional authentication information can be configured manually via custom headers in the GitLab webhook settings.

Two webhook types are used: template repository webhooks and group repository webhooks.

A webhook is automatically created for the template repository. It listens for issue events and push events affecting the main branch. When one of these events occurs, GitLab notifies Moodle that the template repository has been updated. Moodle then propagates these changes to all group repositories. Updates to the practical work instructions issue are applied directly to the corresponding issue in each group repository. At the same time, changes pushed to the main branch are propagated through merge requests, allowing groups to incorporate the latest updates. This mechanism allows teachers and reviewers to modify the practical work even after student repositories have been created.

Webhooks

Use [webhooks](#) to send notifications to external applications in response to events in a GitLab project or group. Consider using an [integration](#) instead.

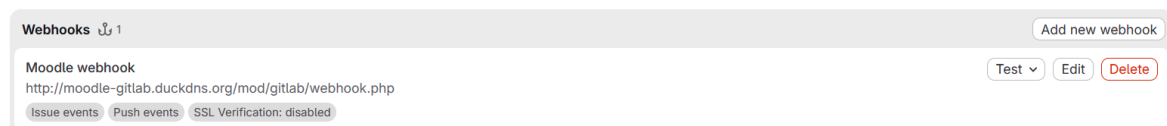


Figure 27: GitLab template repository webhook

A webhook is also created for each group repository to monitor submission merge request events. It listens for close actions on submission merge requests. When a merge request is closed, GitLab notifies Moodle, which identifies the corresponding group submission and updates the submission status. This allows grading-related events to be automatically reflected in Moodle and informs students when their work has been evaluated.

9.11. Protected files

A GitLab CI verification job is automatically created in every repository to detect modifications to protected files.

The job runs during merge request pipelines and compares the files modified by the merge request against a list of protected file patterns.

The `.gitlab/protected-files` file contains one protected file or pattern per line, allowing teachers to define which files should not be modified as part of the assignment.

If a modified file matches one of the protected patterns, the pipeline fails and reports the affected file. This provides students with immediate feedback when they modify a protected file and helps reviewers identify changes that may require additional attention during the evaluation process.

This mechanism is intended as an informational safeguard rather than a security feature. It assists reviewers in identifying potentially sensitive modifications but does not prevent users from modifying the CI configuration itself.

9.12. Custom GitLab host

The integration supports a custom GitLab host by adding a configuration option to specify a GitLab URL other than the default GitLab instance. The configured host is used by the GitLab client when communicating with the REST API, allowing the plugin to work with self-hosted GitLab deployments.

[Activity modules](#) / [gitlab](#)

site



[General](#)

[Users](#)

[Courses](#)

[Grades](#)

[Plugins](#)

[Appearance](#)

[More](#) ▾

gitlab

GitLab host
mod_gitlab | gitlab_host

Default: https://gitlab.com

Enter the base URL of your GitLab instance, for example: https://gitlab.com. Do not include a trailing slash.

Save changes

Figure 28: Plugin custom GitLab host configuration

10. Solution

This chapter presents the final solution delivered by **RAW**. It summarizes the main components of the developed Moodle activity module, the features available to teachers and students, and the resulting GitLab integration.

10.1. Overview

The plugins provide a **RAW** Activity module to automate group management, provisioning of GitLab resources, and other tasks. It also serves as the central point for accessing all important information. All the useful information, such as instructions, reviews, submissions, and starter code, is available in a single GitLab repository per group and is directly accessible in Moodle. Both platforms are kept synchronized using bidirectional communication.

The developed solution provides a Moodle activity module that integrates GitLab into the management of practical assignments. It automates the creation and management of student groups, provisioning GitLab resources, and configuring the associated development workflow, significantly reducing the manual effort required of teachers.

The activity module acts as the central entry point for both teachers and students. All information related to an assignment—including the project description, starter code, repositories, submissions, reviews, and the reference solution—is accessible directly from Moodle.

Each student group is provided with a dedicated GitLab repository that serves as the collaborative workspace for the assignment. In addition, teachers have access to a private template repository for preparing the practical work. This repository contains the starter code distributed to student groups, as well as the reference solution used for evaluation, which can be published to students at any time.

The integration maintains synchronization between Moodle and GitLab through bidirectional communication. Actions performed on either platform, such as repository submissions or evaluations, are propagated automatically, ensuring that both systems remain consistent throughout the assignment lifecycle.

10.2. Teacher Features

The **RAW** activity module provides teachers with a centralized interface for creating and managing assignments. When creating an activity, teachers can define the assignment properties, including its name, description, instructions, submission deadline, maximum group size, and the reviewers responsible for the evaluation.

For each activity, the module automatically provisions a dedicated GitLab group comprising a private template repository and one repository per student group. The repository structure is consistent across all assignments, while repository permissions are automatically configured based on assigned roles. Students receive access only to their group's repository, whereas reviewers are granted the permissions required to evaluate submissions.

The template repository serves as the reference for preparing the practical assignment. Teachers can maintain the starter code, assignment instructions, and reference solution from a single location. Any updates to the starter code or instructions can be propagated to existing group repositories, allowing corrections or improvements to be distributed even after student repositories have been created. Group modifications can also be reviewed via a merge request in the template repository, providing access to all submissions from a single repository. This reduces administrative overhead and simplifies the management of large numbers of student repositories.

The activity dashboard provides an overview of all student groups and their associated repositories. Teachers can create, modify, or remove groups, monitor participation, and access each group's repository directly from Moodle. The dashboard also centralizes submission information, allowing reviewers to determine whether submissions were made before or after the submission deadline, inspect the latest CI/CD pipeline results, review merge requests, retrieve the project source code for local evaluation, and identify groups whose submissions have already been graded.

Once the evaluation is complete, teachers can publish the reference solution and automatically notify students that their work has been reviewed.

10.3. Student Features

Students can create or join groups by accessing the list of available groups. Each group has access to a dedicated GitLab repository, where members can collaborate on the assignment, manage their code, and submit their work. The practical work instructions and due date are also available in a GitLab issue.

Additionally, students are informed when modifications are made to protected files, helping them identify changes that may affect their implementation.

Students can also access assignment due dates on the Moodle calendar and receive reminder notifications to help them manage their workload and submit on time.

Once grading is completed, students are notified on Moodle and can access feedback through GitLab merge requests, allowing them to associate evaluation comments directly with their submitted code.

If teachers have submitted the solution, students can compare their implementation with the expected solution through a GitLab merge request. This allows them to better understand the intended approach and identify possible improvements to their work.

10.4. Integration Flows

The integration relies on a set of automated workflows to provision GitLab resources and react to GitLab hooks. These workflows cover the main stages of the practical assignment lifecycle, from module initialization and group creation to repository synchronization and grading notifications.

The following sequence diagrams illustrate the main integration flows:

10.4.1. Module creation

The module creation flow describes the automated initialization of a practical assignment, including the creation of the GitLab group and template repository, and the configuration of permissions, protected files, webhooks, and Moodle scheduling tasks.

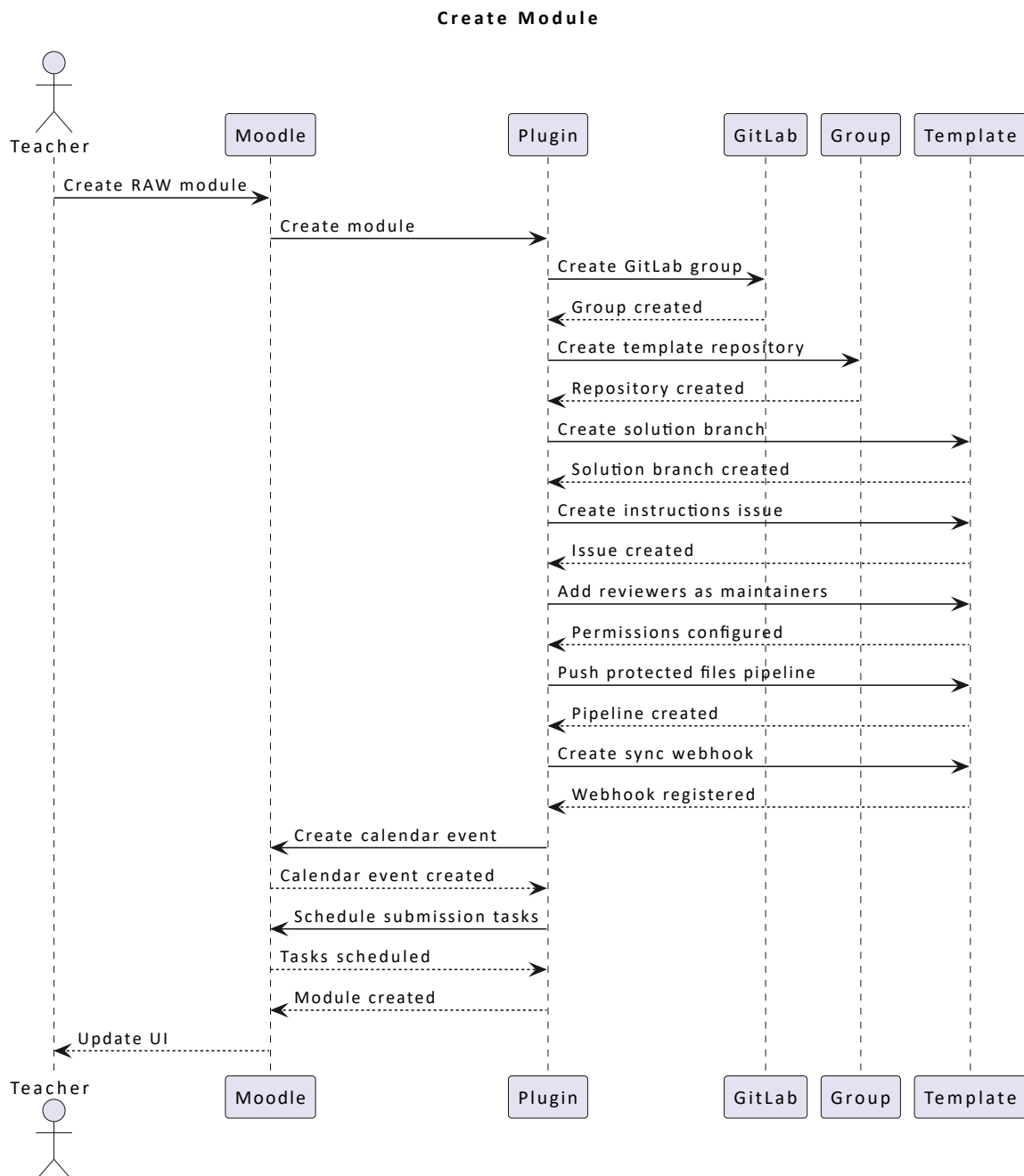


Figure 29: Module creation flow

10.4.2. Group creation

The group creation flow covers provisioning student workspaces, including repository forking, branch configuration, merge request setup, permission assignment, and synchronization of assignment instructions.

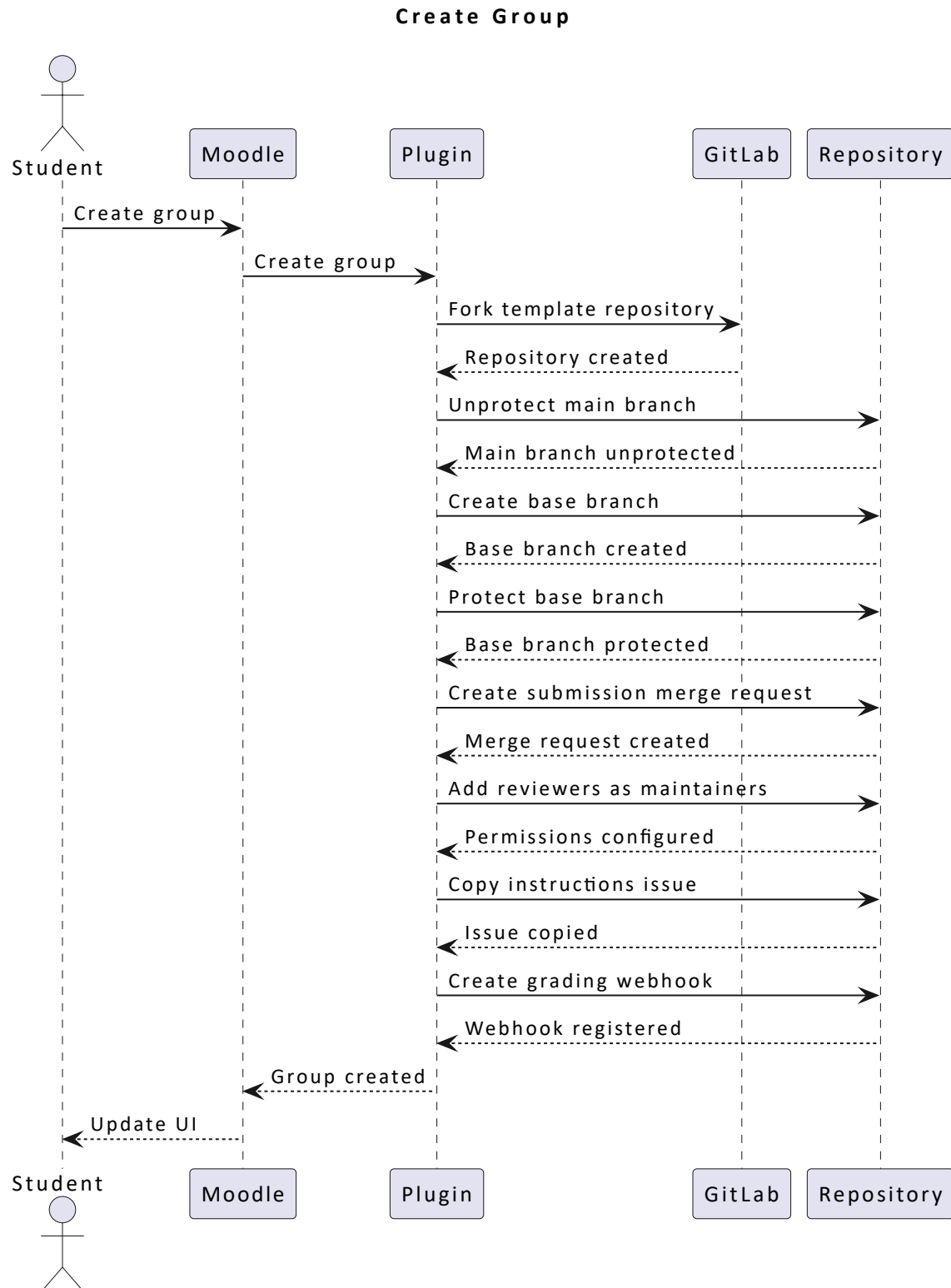


Figure 30: Group creation flow

10.4.3. Template repository webhook

The template repository webhook flow illustrates how updates made to the template repository are propagated to group repositories. It handles synchronization events such as code updates and instruction changes while avoiding duplicate merge requests.

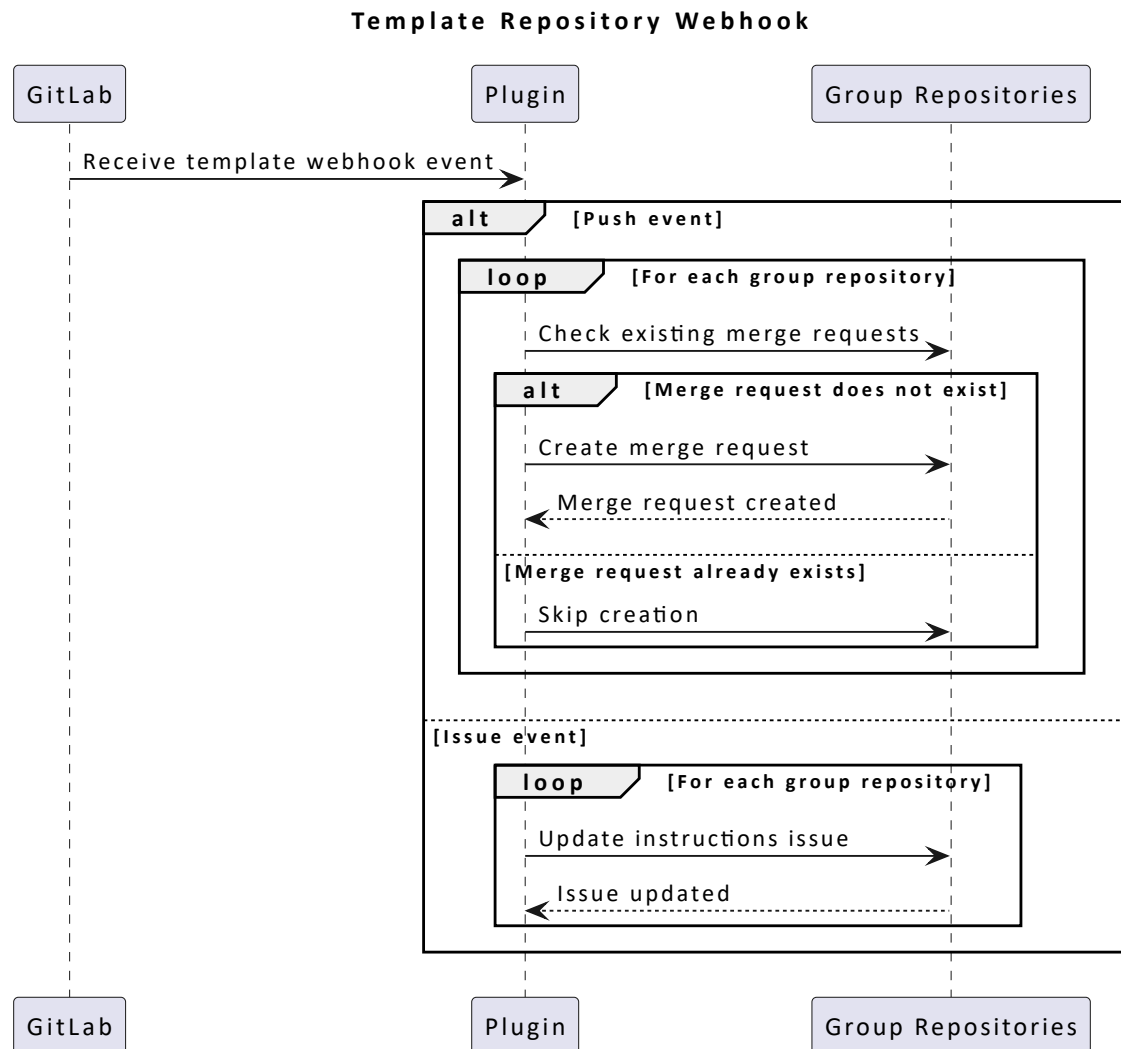


Figure 31: Template repository webhook flow

10.4.4. Group repository webhook

The group repository webhook flow describes the grading notification process triggered by merge request events. Once a submission is reviewed and the merge request is closed, notifications are sent to the corresponding group members.

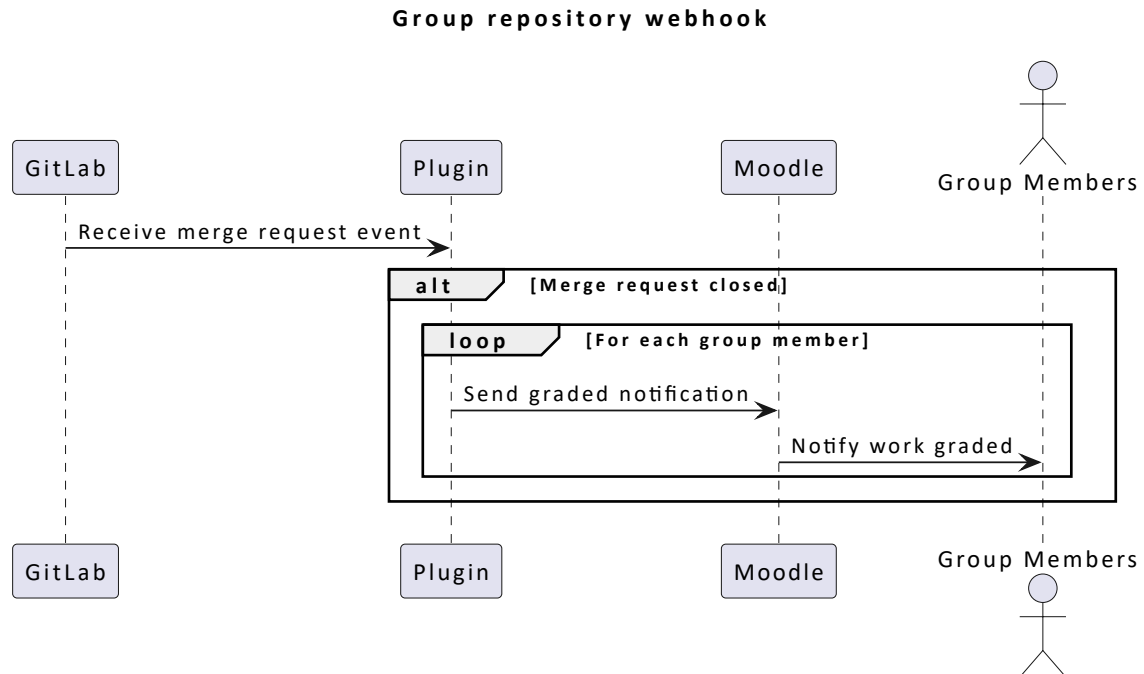


Figure 32: Group repository webhook flow

10.5. Result

RAW achieved a seamless integration between Moodle and GitLab, providing a unified workflow for the creation, management, submission, and evaluation of practical assignments. By automating repository provisioning, synchronization, permission management, and feedback workflows, the system significantly reduces the amount of manual administration required from teachers. The plugin acts as a bridge between the Learning Management System (LMS) and Version Control System (VCS), reducing platform fragmentation and establishing a reference workflow for managing practical assignments.

Overall, the solution standardizes practical assignment management and improves collaboration between students and teachers. It automates repetitive administrative tasks as much as possible while preserving the flexibility and customization required to adapt assignments to different teaching contexts and requirements.

11. Contributions, Limitations and Future Work

11.1. Project Contributions

Before this project, practical assignment management often required manual coordination between Moodle and GitLab, including repository creation, permission configuration, assignment setup, and synchronization of updates. While existing solutions provided partial automation or integration capabilities, they generally lacked deep Learning Management System (LMS) integration or relied on proprietary platforms.

This project extends the existing workflow by introducing an open-source, Moodle-native integration that connects Moodle and GitLab into a unified assignment management process. The proposed solution automates repetitive administrative tasks while preserving the flexibility and customization required for different teaching contexts, avoiding the limitations often introduced by overly rigid automation systems. By reducing platform fragmentation and providing a reference workflow for practical assignment management, the solution improves the consistency and maintainability of computer science education practices.

11.2. Limitations

RAW still has a few limitations due to restrictions in GitLab's permission model. The main examples include:

- Students can trigger actions that they should not be able to. For example, closing the submission merge request in their repository causes the plugin to send a “graded” notification in Moodle for every student in their group, even if the submission has not actually been graded.
- The plugin relies on specific issue and merge request labels to track its state. If these labels are removed or modified manually, the plugin can no longer correctly identify the corresponding issues or merge requests, leading to undefined behaviors.
- The CI job responsible for checking protected files can be modified or removed entirely by students. As a result, the protection mechanism can be bypassed, rendering the check ineffective.

These limitations are not easily addressed because GitLab does not provide fine-grained permissions for individual repository actions. Instead, permissions are managed through predefined user roles, making it difficult to restrict specific actions without also restricting broader repository access.

11.3. Future Work

Several areas could be improved in future iterations of the project. First, additional work is required to strengthen error handling and improve user feedback. Providing clearer error messages and guidance would help users better understand the cause of failures and facilitate troubleshooting.

Another limitation is that the system currently operates synchronously. This can lead to delays while requests are being processed, as users must wait for the operation to complete before receiving a response. An improvement would be to use Moodle built-in ad hoc tasks to provide faster user feedback and to process requests asynchronously in the background.

The Moodle permission management could also be improved. Each action should have its required permissions explicitly defined and validated. Moodle provides a capability-based permission system

that allows fine-grained control through custom capabilities, which should be integrated to improve the plugin's security and modularity.

Finally, future development could focus on extending support to additional Version Control Systems (VCS), such as GitHub. Supporting multiple VCS platforms would increase the solution's flexibility and allow institutions to adopt it across different technical environments.

12. Appendices

12.1. Use of Artificial Intelligence

The [ChatGPT](#) [28] artificial intelligence tool was used during the writing of this thesis for language correction, translation assistance, and improving the overall clarity of the text. It was also used as a support tool during development to identify appropriate Moodle core APIs for specific use cases and to determine their correct usage when the official documentation was incomplete or lacked sufficient details.

The [Grammarly](#) [29] tool was also used to verify grammatical correctness and improve phrasing.

12.2. Planning

See [gantt.xlsx](#)

12.3. Weekly meeting reports

See [reports.pdf](#)

12.4. Source code

[GitHub Repository](#) [30]

12.5. Project documentation

[Documentation](#) [31]

12.6. Kanban Board

[Kanban Board](#) [21]

12.6. Bibliography

- [1] Moodle, “Moodle.” Accessed: Jul. 16, 2026. [Online]. Available: <https://moodle.org/>
- [2] Blackboard, “Blackboard LMS.” Accessed: Jul. 16, 2026. [Online]. Available: <https://www.blackboard.com/solutions/lms>
- [3] Moodle, “The Moodle Story.” Accessed: Jul. 16, 2026. [Online]. Available: <https://moodle.com/about/the-moodle-story/>
- [4] Moodle, “Moodle architecture.” Accessed: Jul. 16, 2026. [Online]. Available: https://docs.moodle.org/dev/Moodle_architecture
- [5] Instructure, “Canvas.” Accessed: Jul. 16, 2026. [Online]. Available: <https://www.instructure.com/canvas>
- [6] PHP Group, “PHP.” Accessed: Jul. 16, 2026. [Online]. Available: <https://www.php.net/>
- [7] GitLab, “GitLab.” Accessed: Jul. 16, 2026. [Online]. Available: <https://about.gitlab.com/>
- [8] GitLab4J, “GitLab4J.” Accessed: Jul. 16, 2026. [Online]. Available: <https://github.com/gitlab4j/gitlab4j-api>
- [9] Stewart, R., “GitLab Haskell.” Accessed: Jul. 16, 2026. [Online]. Available: <https://gitlab.com/robstewart57/gitlab-haskell>
- [10] L. Schauer, R. Stewart, and M. Maarek, “Integrating Canvas and GitLab to enrich learning processes,” in *2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, IEEE, 2024, pp. 180–190. doi: 10.1145/3639474.3640056.
- [11] python-gitlab, “Python GitLab documentation.” Accessed: Jul. 16, 2026. [Online]. Available: <https://python-gitlab.readthedocs.io/>
- [12] GitLab PHP, “GitLab PHP API client.” Accessed: Jul. 16, 2026. [Online]. Available: <https://github.com/GitLabPHP/Client>
- [13] GitHub, “GitHub Classroom transitioning to partners.” Accessed: Jul. 16, 2026. [Online]. Available: <https://github.com/orgs/community/discussions/196615>
- [14] Codio, “Codio.” Accessed: Jul. 16, 2026. [Online]. Available: <https://www.codio.com/>
- [15] Foundation 50, “Classroom 50.” Accessed: Jul. 16, 2026. [Online]. Available: <https://github.com/foundation50/classroom50/wiki>
- [16] RepoBee, “RepoBee.” Accessed: Jul. 16, 2026. [Online]. Available: <https://repobee.org/>
- [17] Classmoji, “Classmoji.” Accessed: Jul. 16, 2026. [Online]. Available: <https://github.com/classmoji/classmoji>
- [18] Moodle, “Encryption.” Accessed: Jul. 16, 2026. [Online]. Available: https://wimski.org/api/5.0/d4/d91/classcore_1_1encryption.html
- [19] Gradescope, “Gradescope.” Accessed: Jul. 16, 2026. [Online]. Available: <https://www.gradescope.com/>
- [20] ANS, “ANS.” Accessed: Jul. 16, 2026. [Online]. Available: <https://www.ans.app/>
- [21] Jouve, L., “Kanban board.” Accessed: Jul. 16, 2026. [Online]. Available: <https://github.com/users/LeonardJouve/projects/3>

- [22] Jouve, L., "Proof of concept." Accessed: Jul. 16, 2026. [Online]. Available: <https://github.com/LeonardJouve/moodle-gitlab-integration/pull/14>
- [23] HashiCorp, "Terraform." Accessed: Jul. 16, 2026. [Online]. Available: <https://developer.hashicorp.com/terraform>
- [24] Red Hat, "Ansible documentation." Accessed: Jul. 16, 2026. [Online]. Available: <https://docs.ansible.com/>
- [25] Authentik, "Authentik." Accessed: Jul. 16, 2026. [Online]. Available: <https://goauthentik.io/>
- [26] Traefik Labs, "Traefik." Accessed: Jul. 16, 2026. [Online]. Available: <https://traefik.io/traefik>
- [27] Moodle, "Cron." Accessed: Jul. 16, 2026. [Online]. Available: <https://docs.moodle.org/502/en/Cron>
- [28] ChatGPT, "Chat GPT." Accessed: Jul. 16, 2026. [Online]. Available: <https://chatgpt.com/>
- [29] Grammarly, "Grammarly." Accessed: Jul. 16, 2026. [Online]. Available: <https://www.grammarly.com/>
- [30] Jouve, L., "Repository." Accessed: Jul. 16, 2026. [Online]. Available: <https://github.com/LeonardJouve/moodle-gitlab-integration>
- [31] Jouve, L., "Project documentation." Accessed: Jul. 16, 2026. [Online]. Available: <https://github.com/LeonardJouve/moodle-gitlab-integration/blob/main/README.md>